

Universidad
Nacional de Rosario



Sistemas operativos II

Índice

1. Conceptos básicos **3**
1.1. ¿Que es un sistema operativo? . . . 3
1.2. Funciones . . . 3
1.3. Evolución de los sistemas operativos . . . 3
1.3.1. Proceso por lotes . . . 3
1.3.2. Sistemas en lotes con dispositivos de carga . . . 3
1.3.3. Sistemas multiprogramados . . . 4
1.3.4. Sistemas de tiempo compartido . . . 4
1.4. Organización de los sistemas operativos . . . 4

2. Relación con el hardware **5**
2.1. Jerarquía de almacenamiento . . . 5
2.2. Interrupciones . . . 5
2.3. Acceso a memoria . . . 5
2.4. Llamadas al sistema . . . 5
2.5. Multiprocesamiento . . . 6

3. Procesos **6**
3.1. Conceptos básicos . . . 6
3.2. Estados de un proceso . . . 6
3.3. Información de un proceso . . . 6
3.4. Hilos . . . 6
3.5. Señales . . . 7
3.5.1. Problemas . . . 7
3.5.2. Señales y excepciones de hardware . . . 7
3.5.3. Funciones para señales . . . 7

4. Planificación de procesos **8**
4.1. Tipos de planificación . . . 8
4.2. Tipos de planificación . . . 9
4.3. Parámetros de respuesta . . . 9
4.4. Algoritmos de planificación . . . 10
4.4.1. FCFS(First come first serve) . . . 10
4.4.2. RR(Round robin) . . . 11
4.4.3. SRR(Selfish round robin) . . . 11
4.4.4. SPN(Shortest process next) . . . 11
4.4.5. PSPN(Preemptive shortest process next) . . . 11
4.4.6. HPRN(Highest penalty ratio next) . . . 11
4.4.7. FB(Multilevel feedback) . . . 12
4.4.8. Lotería . . . 12
4.4.9. PS(Priority scheduling) . . . 12
4.4.10. Métodos dependientes del sistema . . . 13
4.5. Inversión de prioridades . . . 13
4.6. Planificación en tiempo real . . . 13

5. Planificación en Linux **13**
5.1. Propósito . . . 13
5.2. Tipos y estados de tareas . . . 14
5.3. Clases de planificación . . . 14
5.4. Prioridades . . . 14

5.5. CFS(Completely Fair Scheduler) 14
5.5.1. Compromisos 15
5.5.2. CFS y prioridades 15

1. Conceptos básicos

1.1. ¿Que es un sistema operativo?

Cuando hablamos de sistemas operativos en esta cátedra no hacemos referencia al sistema como lo ve o usa el usuario final. Nos interesan los detalles específicos de implementación. Estudiar esto es muy útil pues entender el funcionamiento básico de los sistemas operativos permite diseñar algoritmos y procesos que se ajustan al sistema donde van a ser ejecutados. Más específicamente podemos decir que un sistema operativo es un programa que ejecuta a otros programas, regulando su acceso a los distintos recursos y proveyendo interfaces para interactuar con los mismos.

1.2. Funciones

El sistema operativo es el único programa que interactúa directamente con el hardware de la computadora, sus principales funciones son:

- **Abstracción:** Los programas y procesos no deben tener que preocuparse de los detalles específicos del sistema. No deben tener que preocuparse de cómo acceder al hardware o de sus limitaciones. El sistema operativo se encarga de proporcionar una serie de abstracciones para que los programadores y usuarios puedan enfocarse en sus tareas.
- **Administración de recursos:** Un sistema operativo moderno tendrá a su disposición una gran cantidad de recursos(memoria, almacenamiento, poder de procesamiento, etc); Junto con los distintos procesos del sistema, que competirán por ellos. El sistema debe gestionar los recursos de forma que estos se asignen de forma efectiva y justa acorde a los objetivos del sistema.
- **Aislamiento:** En un sistema multiusuario y multitarea cada proceso y cada usuario no debe percibir que existen otros utilizando el mismo sistema. Idealmente su experiencia es la misma que si estuviera completamente solo. Para esto el sistema debe aislar los procesos y usuarios entre sí, para poder hacer esto es necesario que el sistema operativo utilice hardware específico para dicha protección.

1.3. Evolución de los sistemas operativos

1.3.1. Proceso por lotes

Los predecesores de los sistemas operativos modernos surgen en los años 50 al automatizar el procesamiento de diferentes programas. Los programadores entregaban su lote de tarjetas perforadas a los operadores del sistema que las alimentaban al sistema según correspondiese. En esta primera época el software monitor o de control se encargaba de asistir al operador en la carga de programas y las bibliotecas requeridas. Estos sistemas se fueron complejizando, implementando protecciones que evitaban la corrupción de otros trabajos, o que estos entrasen en un ciclo infinito. Estos monitores implican la modificación del hardware para realizar estas verificaciones, introducciones la característica básica de gestión de recursos.

1.3.2. Sistemas en lotes con dispositivos de carga

Una mejora natural a los sistemas antes mencionados es la invención del spool, un dispositivo de E/S que permite que una computadora de propósito específico, mucho menos costosa lea los datos y los almacene en un medio más rápido, la cinta magnética. Luego el el sistema principal utilizaba esta cinta magnética para E/S, eliminando un cuello de botella significativo.

1.3.3. Sistemas multiprogramados

A lo largo de la ejecución de un programa, inevitablemente, este va a encontrarse a la espera de algún recurso, generalmente procesamiento o E/S. Los sistemas multiprogramados buscaban maximizar el uso efectivo del procesador, ejecutando varios procesos al mismo tiempo. Este cambio de paradigma hizo necesario un cambio importante en el hardware, si bien se esperaba que cada programa fuese razonable con su uso de recursos resultó necesario implementar medidas de protección para garantizar esto.

1.3.4. Sistemas de tiempo compartido

Durante los años 60 la forma de interactuar con las computadoras cambió drásticamente. Los sistemas multiprogramados se convirtieron en sistemas interactivos y multiusuario. Esto llevó a un cambio fundamental sobre la forma en que se controlaban las multitareas:

- **Multitarea cooperativa:** Es la implementación de los sistemas multiprogramados, cada proceso tenía control total del CPU hasta que este hiciese una llamada al sistema o lo entregase voluntariamente. Como consecuencia un programa con bugs o un actor malicioso podía congelar todo el sistema.
- **Multitarea apropiativa:** En los sistemas de tiempo compartido el reloj del sistema interrumpe periódicamente los distintos procesos, entregando forzosamente el control al sistema operativo. Esto evita que un proceso acapare el CPU o congele el sistema.

Además fueron surgiendo de forma gradual las abstracciones que son comunes hoy en día, conceptos como archivos, directorios y librerías del sistema. Al mismo tiempo la velocidad del cambio entre tareas aumentó significativamente, pues en un sistema de tiempo compartido se busca dar la ilusión de uso exclusivo de la computadora.

1.4. Organización de los sistemas operativos

Si bien no hay líneas claras y reglas precisas para clasificar sistemas operativos podemos hacer ciertas distinciones de acuerdo a su estructura:

- **Monolíticos:** Históricamente la mayoría de los sistemas operativos han sido de este tipo. En ellos hay un solo proceso privilegiado(Justamente el sistema operativo) que opera en modo supervisor, donde están todas las rutinas para las tareas que realiza el sistema.
- **Microkernel:** En estos sistemas operativos el núcleo se mantiene lo más pequeño posible, delegando funcionalidades a procesos especiales sin privilegios. La principal ventaja de un sistema de este tipo es la simplificación de una gran cantidad de mecanismos de comunicación, lo que lleva a una mayor velocidad de ejecución. Al mismo tiempo estos sistemas siguen esquemas lógicos más simples, permitiendo implementaciones más elegantes que puede autorepararse con mayor facilidad.
- **Híbridos:** Como se mencionó anteriormente en este espacio no existen delimitaciones claras. Es muy común que un sistema emplee un esquema híbrido, por ejemplo sistemas que son mayormente monolíticos pero delegan ciertas responsabilidades a proceso nivel usuario como un sistema microkernel.

De esta distinción se desprende el hecho de que hay(al menos) dos modos de ejecución:

- **Modo núcleo o kernel:** Al ejecutarse en este modo un proceso tiene acceso a todos los recursos del sistema, se dice que un proceso en este modo es privilegiado. El sistema operativo se ejecuta en este modo.

- **Modo usuario:** Los procesos en este modo tienen restricciones, dependen del sistema operativo para realizar ciertas acciones, que pueden ser denegadas. Los programas nivel usuario se ejecutan de este modo.

2. Relación con el hardware

2.1. Jerarquía de almacenamiento

Las computadoras modernas siguen una arquitectura de Von Neumann, de forma resumida esta se basa en almacenar tanto los programas en ejecución como sus datos en la memoria primaria. Desgraciadamente este sistema tiene un problema importante, llamado el cuello de botella de Von Neumann, este se debe a que la velocidad de los procesadores ha sobrepasado con creces la de la memoria. Una respuesta parcial a este problema es la creación de una jerarquía de la memoria, donde en una misma computadora conviven diferentes memorias de con velocidades, y costos muy distintos. Comenzando con la memoria más rápida tenemos los registros, son parte del procesador, por lo que puede acceder a ellos mientras ejecuta instrucciones. Luego está la memoria caché del procesador, esta no es controlada directamente por el CPU, sino por los controladores de memoria, cargan los datos que el CPU va a necesitar o creen que va a necesitar. Luego tenemos la memoria principal, la RAM, aquí se almacenan los programas en ejecución y sus datos, es la más lenta de todas las memorias mencionadas, así como la más barata y abundante en un sistema común.

2.2. Interrupciones

En el modelo de cómputo actual los procesos y el sistema deben ser capaces de responder a eventos que ocurren alrededor del sistema, son manejados mediante las interrupciones y excepciones. Cuando ocurre un evento que requiere la atención del sistema operativo el hardware encargado de procesarlo genera una interrupción. Esto detiene el proceso que se está ejecutando y la el control al sistema operativo, que ejecuta su rutina de manejo de interrupciones, para luego atender la interrupción. Las interrupciones tienen prioridades, de modo que una interrupción no interrumpa a otra de mayor importancia. El sistema puede elegir ignorar ciertas interrupciones, esto se llama enmascarar, hay ciertas interrupciones que no son enmascarables. La diferencia entre interrupciones y excepciones es su origen, una interrupción es generada por causas externas al sistema, mientras que una excepción es un evento generado por un proceso.

2.3. Acceso a memoria

Originalmente cuando un dispositivo de E/S necesitaba transferir datos a memoria realizaba una interrupción, esta era recibida por el procesador que debía detener su tarea actual para transferir el dato. Este proceso era ineficiente y creaba un cuello de botella. Para solucionar esto las computadoras modernas utilizan controladores DMA, son configurados por el CPU. Una vez configurados se encargan de la transferencia de datos, sin involucrar al procesador; Solo generan interrupciones cuando la transferencia termina u ocurre un error.

2.4. Llamadas al sistema

Como se mencionó anteriormente el sistema aísla los procesos entre sí y los abstrae de los detalles del sistema, esto implica que los procesos no privilegiados no tiene acceso directo a los dispositivos del sistema o ciertas acciones. Cuando un proceso debe realizar alguna acción privilegiada lo hace realizando una llamada al sistema. A grandes rasgos pueden clasificarse en: Control de procesos, manipulación de archivos, manipulación de dispositivos, acceso a información del sistema, comunicación.

2.5. Multiprocesamiento

El multiprocesamiento se da cuando un sistema posee más de un CPU. En este tipo de entornos el conjunto de procesadores se vuelve un recurso más que debe ser gestionado por el sistema operativo. Si bien el término multiprogramación tiende a usarse de forma intercambiable con multiprocesamiento son conceptos fundamentalmente distintos. Un sistema multiprogramado da la ilusión de que se están ejecutando múltiples procesos de forma concurrente, cuando en realidad alterna entre los distintos procesos en ejecución. Un sistema multiprocesado por el otro lado puede realmente ejecutar múltiples procesos de forma concurrente. Vale la pena notar que la mayoría de computadoras modernas tienen un esquema híbrido, donde hay múltiples procesadores, cada uno realizando multiprogramación.

3. Procesos

3.1. Conceptos básicos

En un sistema multiprogramado o de tiempo compartido un proceso es la imagen en memoria de un programa, junto con la información relacionada a su ejecución. Un programa es una entidad pasiva, un conjunto de datos e instrucciones, un proceso es una entidad activa, que proviene de un programa y puede actuar sobre el sistema.

3.2. Estados de un proceso

A lo largo de su vida, un proceso atraviesa una serie de estados de ejecución. Estos son:

- **Nuevo:** La creación de un proceso fue solicitada al sistema operativo, que está creando las estructuras y asignando los recursos necesarios.
- **Listo:** El proceso está listo para comenzar o continuar su ejecución, pero el sistema no le ha asignado ningún procesador.
- **En ejecución:** El proceso está siendo ejecutado por algún procesador.
- **Bloqueado:** El proceso está a la espera de algún evento para poder continuar su ejecución, aun si hubiese un procesador disponible no podría continuar.
- **Zombie:** El proceso terminó su ejecución, pero el sistema operativo debe realizar ciertas operaciones de limpieza antes de que este pueda terminar.
- **Terminado:** El proceso terminó de ejecutarse, sus estructuras están a la espera de ser limpiadas por el sistema operativo.

3.3. Información de un proceso

Toda la información relativa a un proceso es almacenada por el sistema operativo en una estructura llamada *process control block*(PCB). Contiene información sobre el estado del proceso, los registros, scheduling, memoria, E/S, etc.

3.4. Hilos

Es posible que un mismo proceso posea varios marcos de ejecución distintos, estos se conocen como hilos, threads o procesos ligeros. Todos los hilos de un proceso comparten el mismo espacio de memoria, así como los archivos y dispositivos abiertos. Estos se ejecutan de manera individual, cada uno tiene sus propios registros y pila. Los hilos más comunes son los de modo usuario, su ventaja principal es el rendimiento, al no tener que hacer un cambio de contexto para cambiar el hilo que está en ejecución. Sin embargo, esto también implica que cualquier llamada al sistema por parte de uno de

los hilos interrumpe la ejecución de todos los hilos, pues el sistema operativo no tiene conocimiento de los hilos. Por el otro lado existen los hilos nivel kernel, estos son transparentes al sistema y utilizan las funcionalidades que este provee. Esto soluciona el problema con las llamadas al sistema, pero implica una menor eficiencia, pues se debe involucrar al sistema operativo para hacer cambios entre threads.

3.5. Señales

Las señales son una forma de comunicación entre proceso usada en Unix y otros sistemas POSIX. Son notificaciones asíncronas enviadas a un proceso o hilo para notificarlo sobre un evento. Cuando se envía una señal el sistema operativo interrumpe la ejecución normal del destinatario para enviar la señal, esta interrupción puede ocurrir durante cualquier operación no atómica. Si el proceso registró previamente un manejador de señales entonces se ejecuta la rutina correspondiente, de lo contrario se ejecuta el manejador por defecto. Los primeros sistemas tenían un mecanismo de señales muy básico, luego UNIX incorporó un sistema más sofisticado que proveía más información sobre los eventos y permitía una mayor control sobre el mecanismo.

Los comportamientos definidos por defecto al llegar una señal son:

- Terminar el programa (`SIGKILL, SIGINT, SIGALRM`)
- Terminación anormal (`SIGABRT, SIGSEGV, SIGFPE`)
- Ignorar (`SIGCHLD`)
- Detener el programa (`SIGTSTP, SIGSTOP`)
- Continuar, si estaba detenido (`SIGCONT`)

Las señales que terminan un proceso cambian su estado a zombie, las que lo detienen a bloqueado y las que lo continúan a listo.

3.5.1. Problemas

En UNIX el manejo de señales es susceptible a problemas de concurrencia, como son asíncronas puede llegar una señal mientras se está ejecutando el handler de otra señal; También es posible que una señal interrumpa una llamada al sistema, generando un error. Para prevenir problemas adicionales los handlers de señales deben evitar los efectos secundarios: modificar `errno` o atributos globales, usar funciones no-reentrantes, etc.

3.5.2. Señales y excepciones de hardware

La ejecución de un proceso puede resultar en una excepción de hardware, por ejemplo si se intenta dividir por cero o acceder a una dirección de memoria inválida. Si el manejador de excepciones del núcleo no puede resolver el evento entonces envía una señal al proceso para que maneje la excepción.

3.5.3. Funciones para señales

Estas son las funciones definidas segun el estandard POSIX:

- `kill`: Envía la señal especificada al proceso dado, retorna 0 o -1.
  `int kill(pid_t pid, int, sig)`
- `getpid`: Retorna el `PID` del proceso llamante.
  `pid_t getpid(void)`
- `raise`: Autoenvía una señal al proceso que la llama.
  `int raise(int, sig)`

- `signal`: Define el manejador para la señal especificada. También se puede indicar que la señal debe tratarse con el manejador con `SIG_DFL` o que debe ser ignorada con `SIG_IGN`. Las señales `SIGKILL` y `SIGSTOP` no pueden ser interceptadas. Esta función está obsoleta, su implementación varía dependiendo de la versión de UNIX, usar `sigaction`.
  `sighandlert signal(int signum, sighandlert handler)`
- `sigaction`: Versión actualizada de signal, provee más información sobre la señal.
  `int sigaction(int signum, struct sigaction *act, struct sigaction *oldact)`
- `sigprocmask`: Modifica el conjunto de señales bloqueadas para el proceso, se usa para controlar problemas de concurrencia.
  `int sigprocmask (int how, const sigset *set, sigset *oset)`
- `alarm`: Programa un temporizador, cuando llega a cero envía la señal `SIGALRM` al proceso que llamó la función.
  `unsigned int alarm(unsigned int seconds)`

Dado que se ha tomado en algunos parciales se presentan las banderas de `sigaction` y su comportamiento de forma *muy* resumida, todos estos comportamientos son complejos y tienen asteriscos:

- `SA_NOCLDSTOP`: No notificar si el proceso hijo se detiene(stop) o continua(por `SIGCONT`).
- `SA_NOCLDWAIT`: No dejar zombies a los hijos, solo para `SIGCHILD`.
- `SA_NODEFER`: Permite que el handler sea reentrante.
- `SA_ONSTACK`: Utiliza un stack alternativo.
- `SA_RESETHAND`: Vuelve al handler default luego de invocar al handler.
- `SA_RESTART`: Relanza las syscalls interrumpidas por la señal.

4. Planificación de procesos

4.1. Tipos de planificación

La planificación de proceso se refiere a cómo determina el sistema operativo el orden en que le dará acceso al procesador a los distintos procesos que lo solicitan, así como también las políticas para asegurar que el uso del mismo no sea excesivo por parte de ningún proceso. Hay tres tipos principales de planificación:

- **A largo plazo:** Decide qué procesos serán los siguientes en ser iniciados, este tipo de planificación era más frecuente en los sistemas por lotes. En los sistemas interactivos, casi la totalidad de los que se usan hoy en día, este tipo de planificación no se efectúa, dado que normalmente el usuario indica qué procesos iniciar. Hace los pasos entre: `nuevo ->listo`.
- **A mediano plazo:** Decide qué procesos es conveniente bloquear en determinado momento, ya sea por escasez/saturación de algún recurso o porque están realizando una solicitud que no puede satisfacerse. También se encarga de tomar decisiones sobre los procesos a medida que entran y salen del estado bloqueado. Hace los pasos entre `ejecución ->bloqueado` y `bloqueado ->listo`.
- **A corto plazo:** Decide cómo se debe distribuir el uso del CPU momento a momento, se realiza decenas de veces por segundo. Es el encargado de elegir cual de los procesos listos para ejecutar tendrá uso del CPU. Hace los pasos entre `listo ->en ejecución`.

4.2. Tipos de planificación

Los procesos normalmente alternan entre rafagas, en las que realizan principalmente cómputo interno y otras en las que la atención está en hacer E/S. Los procesos largos son los que por mucho tiempo han estado listos o en ejecución, es decir que están en una rafaga limitada por el CPU. Por el otro lado los cortos son los que en este momento están en una rafaga limitada por E/S y requieren atención ocasional del procesador, o tienden a estar bloqueados esperando eventos. Por lo general se le da un tratamiento preferente a los procesos cortos, especialmente a los interactivos.

4.3. Parámetros de respuesta

Mientras que los conceptos de planificación permanecen iguales los tiempos de respuesta cambian constantemente, a medida que evoluciona el hardware. Por lo tanto se emplean unidades abstractas, *ticks* y *quantums* para medir los tiempos de respuesta. El primero representa una fracción de tiempo durante la cual se puede realizar trabajo útil, es decir usar el CPU sin interrupción(Ignorando las interrupciones de E/S). La duración de un tick está definida por una interrupción periódica generada por el temporizador del sistema. En Linux dura un milisegundo, en Windows entre 10 y 15. Los quantums representan el tiempo mínimo que se permite a un proceso el uso del CPU, se define como una cierta cantidad de ticks. En linux es entre 10 y 200, en Windows entre 2 y 12 ticks.

Definimos entonces para un proceso p y un tiempo de ejecución t, las métricas usadas para medir el comportamiento del sistema bajo un planificador dado:

- **Tiempo de respuesta(T):** Tiempo total que requiere el proceso para completar el trabajo pendiente. Se comienza a contar desde que el proceso está listo. Con un único procesador y proceso $T = t$.
- **Tiempo en espera($E = T - t$):** Cuánto tiempo p está listo y esperando para ejecutar, es tiempo perdido. Desde el punto de vista de p se desea que sea cero.
- **Proporción de penalización($P = \frac{T}{t}$):** Proporción en la que fue penalizado el proceso, cuánto del tiempo de respuesta pasó sin ejecutarse.
- **Proporción de respuesta($R = \frac{t}{T}$):** Inverso de P, cuánto tiempo del tiempo de respuesta p se puedo ejecutar.
- **Frecuencia de llegada(α):** Promedio de procesos nuevos en la cola de procesos listos en un rango definido.
- **Frecuencia de servicio requerido(β):** Cuantos procesos terminan en un rango definido.
- **Valor de saturación($\rho = \frac{\alpha}{\beta}$):** Cuando es cero nunca llegan procesos nuevos, si es uno los procesos son despachados al mismo ritmo al que llegan. Si es mayor a uno los procesos llegan más rápido de lo que la computadora puede atenderlos, la cola de procesos listos crece, se degrada la calidad de servicio.
- **Tiempo de kernel:** Tiempo que pasa el sistema en espacio kernel, no se cuenta al calcular el tiempo de CPU de un proceso.
- **Tiempo de sistema:** Tiempo que pasa un proceso en espacio kernel, cuando el sistema está atendiendo sus llamadas al sistema.
- Tiempo de usuario: Tiempo que pasa un proceso en modo usuario, es decir ejecutando su programa.
- **Tiempo de uso del procesador:** Tiempo durante el cual el CPU ejecuta instrucciones del proceso, sean en modo usuario o kernel.

- **Tiempo idle:** Tiempo en el que la cola de procesos está vacía y el CPU no está ejecutando instrucciones. Utilización del CPU: Porcentaje del tiempo en el que el CPU está realizando trabajo útil, en sistemas reales está entre 40 y 90 %.

4.4. Algoritmos de planificación

Analizamos ahora los objetivos de la planificación a corto plazo así como algunos algoritmos que intentan resolver el problema. A grandes rasgos los objetivos de este tipo de planificación(Notando que pueden ser contradictorios) son:

- Ser justo, debe tratar de la misma forma a todos los procesos con características similares y no postergar ninguno de ellos indefinidamente.
- Maximizar el rendimiento, dar servicio a la mayor parte de procesos por unidad de tiempo.
- Ser predecible, un mismo trabajo debe tomar aproximadamente la misma cantidad de tiempo sin importar la carga del sistema.
- Minimizar la sobrecarga, el tiempo que el algoritmo gasta en burocracia debe mantenerse al mínimo.
- Equilibrar el uso de recursos, favorecer a los procesos que utilizan recursos utilizados y penalizar a los que utilicen recursos sobreutilizados.
- Enviar la inanición, aumenta la prioridad de procesos más viejos, permitiendo que se ejecuten en tiempo y forma.
- Favorecer el uso esperado del sistema, en un sistema interactivo maximizar la prioridad de procesos que sirvan a las solicitudes iniciadas por los usuarios.
- Dar preferencia a los procesos que podrían bloquear el sistema, si un recurso de baja prioridad está utilizando un recurso del sistema por el que otros procesos están esperando, favorecer a que este termine para que libere el recurso.
- Favorecer a los procesos con comportamiento deseable, si un proceso causa muchas demoras se lo puede penalizar para evitar que se degrade el rendimiento global del sistema.
- Degradarse suavemente, si $\rho > 1$ se debe tratar de responder con la menor penalización a procesos preexistentes al momento de exceder el umbral.

4.4.1. FCFS(First come first serve)

El esquema más simple posible, es un mecanismo cooperativo, cada proceso se ejecuta en el orden en que llegó, hasta que suelta el control. El despachador es una cola *FIFO*. Notas:

- Minimiza la cantidad de cambios de contexto.
- Reduce la sobrecarga administrativa(Elegir al siguiente es O(1)).
- No requiere hardware de apoyo.
- Favorece a los procesos largos.
- No utiliza información sobre los procesos.
- Los últimos en llegar tienen un comportamiento inaceptable.
- Es inadecuado para trabajos interactivos.

4.4.2. RR(Round robin)

Intenta dar una buena respuesta para procesos cortos y largos, es un mecanismo apropiativo. Cada proceso en la lista de procesos listos solo puede ejecutarse durante una cierta cantidad de *quantums* (q), luego será interrumpido y puesto al final de la lista si es que no ha terminado. El valor de q puede ajustarse, con q arbitrariamente grande se vuelve **FCFS**, idealmente el *quantum* debe mantenerse inferior a la duración promedio de 80 % de los procesos.

4.4.3. SRR(Selfish round robin)

Versión egoísta de **RR**, busca favorecer a los procesos que ya se han ejecutado, a costa de los recién llegados. Utiliza dos colas, una de procesos nuevos y otra de procesos aceptados, los procesos ingresan a la cola de procesos nuevos, avanzan a la cola de aceptados cuando su prioridad alcanza la de algún proceso aceptado. Los elementos aceptados son manejados por un esquema **RR**. El incremento de las prioridades es controlado por dos parámetros a y b, determinan en cuánto aumenta la prioridad de los procesos en la cola de procesos nuevos y aceptados, respectivamente. Notas:

- Si $\frac{b}{a} < 1$ todo proceso nuevo eventualmente será aceptado.
- Si $\frac{b}{a} > 1$ se convierte en **FCFS**.
- Si $\frac{b}{a} = 0$ se convierte en **RR**.
- Utiliza información intrínseca.
- Favorece a los procesos largos sin bloquear completamente a los cortos.
- Los parámetros se deben ajustar correctamente para un buen funcionamiento.

4.4.4. SPN(Shortest process next)

Cuando no se puede implementar multitarea apropiativa pero se requiere un algoritmo más justo se puede usar información sobre lo que un proceso va a demorar en el futuro. Cómo obtener esta información es muy difícil se busca caracterizar las necesidades del proceso analizando como se ha comportado. Se utiliza un promedio exponencial: $e_p^{'} = f.e_p + (1 - f)q$ donde q es los *quantums* que p utilizó en su última ejecución, se elije el proceso con menor e_p. Notas:

- Favorece los procesos cortos.
- Un proceso largo puede sufrir inanición, especialmente si ρ es cercano o mayor a 1.
- Utiliza información intrínseca.

4.4.5. PSPN(Preemptive shortest process next)

Utiliza el mismo mecanismo que **SPN**, con la diferencia de que es apropiativo. Si llega un nuevo proceso con e_p menor al del proceso en ejecución el proceso en ejecución será reemplazado. Obtiene mejores promedios que **SPN** de forma consistente pues mantiene la lista corta al despachar primero los procesos cortos

4.4.6. HPRN(Highest penalty ratio next)

Si no se dispone de multitarea apropiativa los mecanismos **FCFS** y **SPN** no son ideales, este algoritmo intenta ser más balanceado. Cada proceso tiene un índice de penalización P, con un valor inicial de 1. Cada vez que un proceso es obligado a esperar por un tiempo w a otro proceso, su valor de P se actualiza a $P = \frac{w+t}{t}$. El proceso con mayor P se elige como activo. Notas:

- Si $\rho < 1$ ningún proceso sufre de inanición.
- Es costoso calcular P para cada proceso, especialmente si la lista es larga.
- Utiliza información intrínseca.

4.4.7. FB(Multilevel feedback)

Utiliza varias colas, se utiliza mucho hoy en día. Se tienen múltiples colas, el despachador elige el proceso al frente de la cola de mayor prioridad. Tras un número predeterminado de ejecuciones la prioridad de un proceso es degradada, se lo pasa a la cola de prioridad inmediatamente inferior, si es que existe. El tiempo de ejecución permitido para un proceso puede ajustarse de acuerdo a la cola en la que esté. Notas:

- Implementa multitarea apropiativa.
- Con los ajustes adecuados es mejor que **RR**.
- Se puede utilizar *aging* para evitar inanición, evitando que procesos pasen demasiado tiempo sin ejecutarse.
- Utiliza información intrínseca.
- Cada cola puede utilizar un esquema diferente para elegir el próximo proceso.
- Es relativamente complejo de implementar, especialmente si se permiten distintos esquemas.

4.4.8. Lotería

En este esquema cada proceso tiene un número de boletos, cada boleto representa una posibilidad de salir ganador. Para elegir al próximo proceso se elige un número al azar y se otorga el siguiente quantum al proceso ganador. La cantidad de boletos de un proceso es constante. Notas: Las prioridades se representan mediante la cantidad de boletos, a más boletos mayor prioridad. Considera que los procesos pueden cooperar entre sí, por ejemplo si un proceso A espera un recurso de B, puede donar sus boletos para aumentar las probabilidades de que B sea ejecutado. Notas:

- Implementa multitarea apropiativa.
- Evita inanición, pues todo proceso esperando ejecutarse puede ser elegido, y eventualmente lo será.
- Presenta variabilidad a la hora de asignar el CPU.
- No es determinista.

4.4.9. PS(Priority scheduling)

Este esquema puede o no ser apropiativo. En ambos casos se elige el proceso de mayor prioridad para ejecutar, estas prioridades pueden ser internas o externas, dinámicas o estáticas. El caso más sencillo es el no apropiativo, se espera a que el proceso en ejecución termine para elegir su reemplazo. Por el otro lado, en el caso apropiativo el proceso en ejecución será interrumpido si llega o se vuelve listo un proceso de mayor prioridad. Notas:

- Puede producir inanición de los procesos de baja prioridad, se puede mitigar este problema usando *aging*.
- Es sencillo de implementar.

4.4.10. Métodos dependientes del sistema

Si bien no existe un esquema de planificación ideal se puede tomar una decisión en base a las características del sistema:

- Si en promedio, los procesos son cortos y $\rho < 1$ se deberían utilizar métodos con poca sobrecarga administrativa, como **FCFS**, **SPN** o **RR** con q grande.
- Si la cola de tareas es muy larga se debe usar **RR** con q pequeño o **PSPN**, que garantizan una buena distribución del CPU.
- Siempre es una buena opción utilizar **RR**, donde q se actualiza periodicamente dependiendo de la cantidad de procesos. Si hay pocos procesos se minimiza la cantidad de cambios de contexto con q grande.

4.5. Inversión de prioridades

Este problema ocurre cuando un proceso que necesita acceder a un recurso debe esperar porque este está siendo usado por un proceso de menor prioridad. Este problema puede mitigarse de dos formas:

- **Techo de prioridad:** Se le asigna una prioridad máxima al recurso compartido, la más alta entre cualquier proceso que pueda acceder a él. Cuando un proceso adquiere el recurso su prioridad se eleva temporalmente a la máxima, cuando libera el recurso, su prioridad se restaura al valor original.
- **Herencia de prioridades:** El proceso de menor prioridad, hereda temporalmente la prioridad del proceso esperando al recurso. Cuando libera el recurso su prioridad se restaura al valor original.

4.6. Planificación en tiempo real

Hay procesos que requieren garantías de tiempo, necesitan un cierto tiempo de ejecución antes de un tiempo límite, se conocen como procesos de tiempo real. Los algoritmos vistos hasta el momento no son adecuados para este tipo de procesos. La opción ideal para manejar procesos de tiempo real es el uso de tiempo real duro, donde el sistema puede garantizar que un proceso tendrá el suficiente tiempo de ejecución en un intervalo dado; No es posible hacer esto en sistemas de propósito general con hardware estándar. Para solventar estas necesidades en sistemas de uso general se usa tiempo real suave, donde los procesos de tiempo real reciben un tratamiento prioritario sobre los procesos comunes, generalmente se implementa con un esquema similar a retroalimentación multinivel, con las siguientes peculiaridades:

- La cola de tiempo real recibe prioridad sobre todas las otras.
- La prioridad de un proceso de tiempo real no se degrada.
- Los demás procesos nunca llegan a la cola de tiempo real por un proceso automático.
- La latencia de despacho debe ser mínima.

5. Planificación en Linux

5.1. Propósito

El planificador de Linux tiene dos objetivos principales. El primero es poder adaptarse lo mejor posible a cualquier entorno de uso, sin requerir demasiado poder de cómputo, pero logrando buen rendimiento. El segundo es optimizar el rendimiento de los procesadores pero sin dejar de lado a las tareas interactivas, lo que implica que las tareas vinculadas a la E/S no pueden ser demoradas por mucho tiempo.

5.2. Tipos y estados de tareas

Basado en los objetivos anteriores Linux intenta hacer una distinción clara basada en el tipo de las tareas:

- **Interactivas:** Suelen estar en estado bloqueado la mayor parte del tiempo, esperando algún evento externo. Cuando se recibe el evento se debe reaccionar lo más rápido posible.
- **Batch:** Hacen un uso intensivo del CPU, pueden permitirse un poco de demora.

El ciclo de una tarea es similar al de cualquier sistema UNIX, con algunas diferencias:

- `TASK_RUNNING`: Esta lista *o* en ejecución, notar que se usa el mismo valor para ambas.
- `TASK_INTERRUPTABLE`: Está bloqueada, pero se le pueden retirar recursos.
- `TASK_UNINTERRUPTABLE`: Está bloqueada, no se le pueden retirar recursos, generalmente se utiliza para funciones críticas y *syscalls*.
- `TASK_STOPPED`: Bloqueada por una señal.
- `TASK_TRACED`: Bloqueada, esta siendo depurada en un *debugger*.
- `EXIT_ZOMBIE`: Zombie.
- `EXIT_DEAD`: Terminada.

El planificador descrito a continuación se encarga de las tareas en estado `TASK_RUNNING`.

5.3. Clases de planificación

Como se vio anteriormente las tareas de tiempo real requieren una atención y tiempo de respuesta mayor a los procesos regulares, para esto utilizan dos colas especiales:

- `SCHED_FIFO`: Las tareas en esta clase solo pueden ser reemplazadas por tareas de mayor prioridad, de lo contrario se ejecutan hasta terminar o bloquearse.
- `SCHED_RR`: Se utiliza un esquema **RR** con *quantum* fijo.

El resto de las tareas tienen la clase `SCHED_OTHER`, analizaremos principalmente este tipo de tareas.

5.4. Prioridades

Cada tarea tiene una propiedad estática que se puede ajustar con las funciones `nice` y `set_priority`. Los procesos hijos e hilos creados por una tarea heredan su prioridad. Internamente el planificador le asigna una prioridad dinámica a cada tarea. Está basada en la prioridad estática, pero varía en función del tipo de tarea y el estado del sistema.

Linux utiliza prioridades entre 0 y 139, donde 0 es la mayor, los valores entre 0 y 99 están reservados para tareas de tiempo real; Es decir tareas cuya clase es `SCHED_FIFO` o `SCHED_RR`.

5.5. CFS(Completely Fair Scheduler)

El scheduler usado por Linux actualmente, actualmente se está preparando su reemplazo **EEVDF**. Este algoritmo pretende simular el comportamiento de una CPU ideal, divide el poder de cómputo equitativamente entre todas las tareas listas, dado que en un CPU real no pueden ejecutarse dos tareas en simultáneo se opta por intentar dividir el tiempo de uso del CPU.

Para hacer esto se usa una técnica simple basada en el conteo, llamada *virtual runtime*, en la práctica indica cuanto tiempo de ejecución tuvo una tarea, normalizado sobre el número total de tareas. Cuando ocurre una decisión de planificación **CFS** selecciona el proceso con menor *virtual runtime* para ejecutarse a continuación. Las tareas en estado *runnable* se almacenan en run *Red Black Tree*, que permite acceso de orden $O(log(n))$.

5.5.1. Compromisos

Para decidir la frecuencia de cambios de contexto se debe hacer un compromiso. Cambios de contexto más frecuentes mejoran la equidad pero degradan el rendimiento, por el contrario menos cambios llevan a menor equidad pero mejor rendimiento.

Este compromiso se gestiona mediante diversos parámetros de control, el más importante es `sched_latency`, es el tiempo total en que todas las tareas listas deberían ejecutarse al menos una vez. **CFS** divide este valor por la cantidad de procesos para obtener un `time_slice`, cuánto tiempo debe ejecutarse cada proceso, asegurando asignación de CPU equitativa. Si hay demasiados procesos en ejecución `time_slice` se volverá muy pequeño, causando demasiados cambios de contexto, para solucionar esto se le asigna una cota inferior a `time_slice`, se llama `min_granularity`.

Para tomar decisiones **CFS** es activado periódicamente mediante un temporizador. Cuando esto ocurre se determina si el proceso actual ha alcanzado el final de su ejecución. Dado que **CFS** lleva un seguimiento preciso del *virtual runtime* de todos los procesos, termina aproximando un reparto ideal de la CPU, incluso con los compromisos mencionados.

5.5.2. CFS y prioridades

El algoritmo descrito hasta el momento no considera las prioridades de las tareas. Para hacer esto **CFS** modifica el `time_slice` de cada proceso mediante un peso basado en la prioridad del mismo, a mayor peso mayor será el `time_slice`. Notar que la relación entre peso y prioridad no es lineal, sino logarítmica. El cálculo del *virtual runtime* es adaptado de una forma similar, se toma el tiempo real de ejecución del proceso y se lo escala de manera inversa según el peso de su prioridad.